MADES Algorithm Program Spring Qualifications - Practice Contest - Editorial

Marmara University Developer Society - Competitive Programming Club

March 2026

A. Presents

The Logic

Let's say we have an array p. If person i gave a gift to person p_i, then in our answer we should swap p_i and i since person p_i received a gift from person i.

We can create an `ans` array. As we read the input in a loop, we simply set `ans[`p_i`-1] = i+1`. Since the constraints are small ($1 \leq n \leq 100$), this $O(N)$ approach is perfectly optimal.

Code Implementation (C++)

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n; // read the number of friends from stdin
    int ar[n], ans[n];
    for(int i = 0; i < n; i++) {
        cin >> ar[i]; // taking the space-separated input from stdin
        ans[ar[i]-1]=i+1; // -1/+1  to handle zero-based indexing
    }
    for(int i = 0;i < n; i++) cout<<ans[i]<<" "; // printing the answer
    to stdout

    return 0;
}
```

Code Implementation (Python 3)

```python
# Read the number of friends from standard input
n = int(input())

# Read the space-separated list of gift receivers and map to integers
p = list(map(int, input().split()))
ans = [0] * n

for i in range(n):
    # -1/+1 to handle zero-based indexing
```

```
    ans[p[i] - 1] = i + 1

# Print the array elements separated by spaces to standard output
print(*ans)
```

Code Implementation (Java)

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Read the number of friends
        int n = sc.nextInt();
        int[] ans = new int[n];

        for (int i = 0; i < n; i++) {
            // Scanner automatically handles space-separated inputs
            int p = sc.nextInt();
            ans[p-1] = i+1;
        }

        for (int i = 0; i < n; i++) {
            // Print the answer to standard output
            System.out.print(ans[i] + " ");
        }
    }
}
```

Code Implementation (C#)

```
using System;

class Program {
    static void Main() {
        // Read the number of friends
        int n = int.Parse(Console.ReadLine());

        // Read the whole line and split it by spaces into an array
        string[] inputs = Console.ReadLine().Split();
        int[] ans = new int[n];

        for (int i = 0; i < n; i++) {
            // Parse each space-separated string into an integer
            int p = int.Parse(inputs[i]);
            ans[p-1] = i+1;
        }

        for (int i = 0; i < n; i++) {
            // Print to standard output
            Console.Write(ans[i] + " ");
        }
    }
}
```

B. IQ-Test

The Logic

Since the grid is very small (4×4), we can brute-force the solution. There are exactly nine possible 2×2 subgrids inside a 4×4 grid.

For a 2×2 grid to become a single color with *at most one* change, it must already have **3 or 4 cells of the same color**. So, we can loop through all 9 possible top-left corners of these 2×2 grids, count the number of black and white cells inside them, and check if either count is ≥ 3. If we find even one such 2×2 grid, the answer is "YES". If we check all 9 and none of them work, the answer is "NO".

Code Implementation (C++)

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<string> grid(4);
    // Read 4 lines of strings representing the grid
    for (int i = 0; i < 4; i++) cin >> grid[i];

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            int black = 0, white = 0;

            for (int r = i; r < i + 2; r++) {
                for (int c = j; c < j + 2; c++) {
                    if (grid[r][c] == '#') black++;
                    else white++;
                }
            }

            if (black >= 3 || white >= 3) {
                // Print YES to stdout and exit early
                cout << "YES\n";
                return 0;
            }
        }
    }
    // Print NO if no valid 2x2 square was found
    cout << "NO\n";
    return 0;
}
```

Code Implementation (Python 3)

```python
# Read 4 lines of string input from stdin using list comprehension
grid = [input() for _ in range(4)]

for i in range(3):
    for j in range(3):
        black = 0
        white = 0

```

```
        for r in range(i, i + 2):
            for c in range(j, j + 2):
                if grid[r][c] == '#':
                    black += 1
                else:
                    white += 1

        if black >= 3 or white >= 3:
            # Print YES and terminate the program
            print("YES")
            exit()

# Print NO if no valid 2x2 square was found
print("NO")
```

Code Implementation (Java)

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] grid = new String[4];

        // Read 4 consecutive strings from standard input
        for (int i = 0; i < 4; i++) {
            grid[i] = sc.next();
        }

        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                int black = 0, white = 0;

                for (int r = i; r < i + 2; r++) {
                    for (int c = j; c < j + 2; c++) {
                        if (grid[r].charAt(c) == '#') black++;
                        else white++;
                    }
                }

                if (black >= 3 || white >= 3) {
                    // Print YES to stdout and exit early
                    System.out.println("YES");
                    return;
                }
            }
        }
        // Print NO if no valid 2x2 square was found
        System.out.println("NO");
    }
}
```

Code Implementation (C#)

```
using System;

```

```
class Program {
    static void Main() {
        string[] grid = new string[4];

        // Read 4 lines representing the grid from standard input
        for (int i = 0; i < 4; i++) {
            grid[i] = Console.ReadLine();
        }

        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                int black = 0, white = 0;

                for (int r = i; r < i + 2; r++) {
                    for (int c = j; c < j + 2; c++) {
                        if (grid[r][c] == '#') black++;
                        else white++;
                    }
                }

                if (black >= 3 || white >= 3) {
                    // Print YES to stdout and exit early
                    Console.WriteLine("YES");
                    return;
                }
            }
        }
        // Print NO if no valid 2x2 square was found
        Console.WriteLine("NO");
    }
}
```

C. Adjacent Letters

The Logic

Since we can only delete characters in pairs of two, any character that remains at the end must have had an *even* number of characters deleted to its left, and an *even* number of characters deleted to its right. In a 0-indexed string, this means the target character c must be located at an **even index** (0, 2, 4, etc.). We simply scan the string, looking only at the even indices, to see if c exists there.

Code Implementation (C++)

```cpp
#include <iostream>
using namespace std;

void solve() {
    string s; char c;
    // cin reads strings and chars, skipping whitespaces automatically
    cin >> s >> c;
    for (int i = 0; i < s.length(); i += 2) {
        if (s[i] == c) {
            cout << "YES\n";
            return;
        }
    }
    cout << "NO\n";
}

int main() {
    int t;
    // Read number of test cases
    cin >> t;
    while (t--) solve();
    return 0;
}
```

Code Implementation (Python 3)

```python
# Read the number of test cases
t = int(input())

for _ in range(t):
    # Read the string and the target character
    s = input()
    c = input()

    found = False
    for i in range(0, len(s), 2):
        if s[i] == c:
            found = True
            break

    # Print output based on the boolean result
    print("YES" if found else "NO")
```

Code Implementation (Java)

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Read the number of test cases
        int t = sc.nextInt();

        while (t-- > 0) {
            // Read string and character directly from standard input
            String s = sc.next();
            char c = sc.next().charAt(0);

            boolean found = false;
            for (int i = 0; i < s.length(); i += 2) {
                if (s.charAt(i) == c) {
                    found = true;
                    break;
                }
            }
            // Print output based on the boolean result
            System.out.println(found ? "YES" : "NO");
        }
    }
}
```

Code Implementation (C#)

```csharp
using System;

class Program {
    static void Main() {
        // Read the number of test cases
        int t = int.Parse(Console.ReadLine());

        while (t-- > 0) {
            // Read string and character from standard input
            string s = Console.ReadLine();
            char c = Console.ReadLine()[0];

            bool found = false;
            for (int i = 0; i < s.Length; i += 2) {
                if (s[i] == c) {
                    found = true;
                    break;
                }
            }
            // Print output based on the boolean result
            Console.WriteLine(found ? "YES" : "NO");
        }
    }
}
```

D. Eating Candies

The Logic

This requires the **Two Pointers** technique. We place a pointer L at the start and R at the end. We maintain the current sum of weights for Alice (`sumA`) and Bob (`sumB`).

- If `sumA < sumB`, Alice needs more weight, so move L right.
- If `sumB < sumA`, Bob needs more weight, so move R left.
- If `sumA == sumB`, we record the total number of candies eaten as our current best answer, and then move either pointer to keep searching for larger sets.

Code Implementation (C++)

```cpp
#include <iostream>
#include <vector>
using namespace std;

void solve() {
    int n;
    cin >> n; // read number of candies
    vector<int> w(n);

    // read space-separated candy weights
    for (int i = 0; i < n; i++) cin >> w[i];

    int L = 0, R = n - 1;
    long long sumA = w[L], sumB = w[R];
    int max_candies = 0;

    while (L < R) {
        if (sumA == sumB) {
            max_candies = (L + 1) + (n - R);
            sumA += w[++L];
        } else if (sumA < sumB) {
            sumA += w[++L];
        } else {
            sumB += w[--R];
        }
    }
    // print max candies to standard output
    cout << max_candies << "\n";
}

int main() {
    int t;
    cin >> t; // read number of test cases
    while (t--) solve();
    return 0;
}
```

Code Implementation (Python 3)

```python
# Read number of test cases
t = int(input())
```

```python

for _ in range(t):
    # Read number of candies
    n = int(input())
    # Read space-separated weights into a list
    w = list(map(int, input().split()))

    L, R = 0, n - 1
    sumA, sumB = w[L], w[R]
    max_candies = 0

    while L < R:
        if sumA == sumB:
            max_candies = (L + 1) + (n - R)
            L += 1
            sumA += w[L]
        elif sumA < sumB:
            L += 1
            sumA += w[L]
        else:
            R -= 1
            sumB += w[R]

    # Print the result to stdout
    print(max_candies)
```

Code Implementation (Java)

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Read number of test cases
        int t = sc.nextInt();

        while (t-- > 0) {
            int n = sc.nextInt();
            int[] w = new int[n];
            // Read space-separated weights using Scanner
            for (int i = 0; i < n; i++) w[i] = sc.nextInt();

            int L = 0, R = n - 1;
            long sumA = w[L], sumB = w[R];
            int max_candies = 0;

            while (L < R) {
                if (sumA == sumB) {
                    max_candies = (L + 1) + (n - R);
                    L++;
                    sumA += w[L];
                } else if (sumA < sumB) {
                    L++;
                    sumA += w[L];
                } else {
                    R--;
```

```
                    sumB += w[R];
                }
            }
            // Print to standard output
            System.out.println(max_candies);
        }
    }
}
```

Code Implementation (C#)

```
using System;

class Program {
    static void Main() {
        // Read number of test cases
        int t = int.Parse(Console.ReadLine());

        while (t-- > 0) {
            int n = int.Parse(Console.ReadLine());

            // Read space-separated weights and parse them
            string[] inputs = Console.ReadLine().Split();
            int[] w = new int[n];
            for (int i = 0; i < n; i++) w[i] = int.Parse(inputs[i]);

            int L = 0, R = n - 1;
            long sumA = w[L], sumB = w[R];
            int maxCandies = 0;

            while (L < R) {
                if (sumA == sumB) {
                    maxCandies = (L + 1) + (n - R);
                    L++;
                    sumA += w[L];
                } else if (sumA < sumB) {
                    L++;
                    sumA += w[L];
                } else {
                    R--;
                    sumB += w[R];
                }
            }
            // Print the result to stdout
            Console.WriteLine(maxCandies);
        }
    }
}
```

E. K-th Not Divisible by n

The Logic

If you try to iterate on the number line and count the numbers you can take by skipping when the current number i is divisible by n, you will get the verdict *Time Limit Exceeded*. Since that approach runs in $O(k)$, and most judging systems can compute around 10^8 operations in 1 second, an $O(k)$ answer does **not** pass for $k < 10^9$.

We can solve this in $O(1)$ time using math. Every time we count $n-1$ valid numbers, we skip exactly 1 number (which is the multiple of n). To find the k-th number, we just need to figure out how many multiples of n we had to skip to get there, and add that skip count to k.

Why do we use $k-1$ instead of k in the formula? If we just calculated $k/(n-1)$, we would accidentally add an extra skip right on the boundary when k is an exact multiple of $n-1$. By subtracting 1 first, we shift the calculation so the extra skip is only added *after* we fully pass the group of $n-1$ valid numbers.

The number of skips required is exactly $\lfloor (k-1)/(n-1) \rfloor$. So, the final answer is $k + \lfloor (k-1)/(n-1) \rfloor$.

Code Implementation (C++)

```cpp
#include <iostream>
using namespace std;

void solve() {
    long long n, k;
    // cin reads space-separated long longs automatically
    cin >> n >> k;

    long long skips = (k - 1) / (n - 1);

    // Print the calculated answer to stdout
    cout << k + skips << "\n";
}

int main() {
    int t;
    cin >> t; // Read number of test cases
    while (t--) solve();
    return 0;
}
```

Code Implementation (Python 3)

```python
# Read the number of test cases
t = int(input())

for _ in range(t):
    # Read n and k from a single space-separated line
    n, k = map(int, input().split())

    # Calculate skips using integer division
    skips = (k - 1) // (n - 1)
```

```
    # Print the answer to standard output
    print(k + skips)
```

Code Implementation (Java)

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;

        // Read the number of test cases
        int t = sc.nextInt();

        while (t-- > 0) {
            // Read n and k directly handling spaces
            long n = sc.nextLong();
            long k = sc.nextLong();

            long skips = (k - 1) / (n - 1);

            // Print output
            System.out.println(k + skips);
        }
    }
}
```

Code Implementation (C#)

```csharp
using System;

class Program {
    static void Main() {
        // Read the number of test cases
        int t = int.Parse(Console.ReadLine());

        while (t-- > 0) {
            // Read space-separated values
            string[] inputs = Console.ReadLine().Split();

            // Parse long values
            long n = long.Parse(inputs[0]);
            long k = long.Parse(inputs[1]);

            long skips = (k - 1) / (n - 1);

            // Print the calculated answer
            Console.WriteLine(k + skips);
        }
    }
}
```